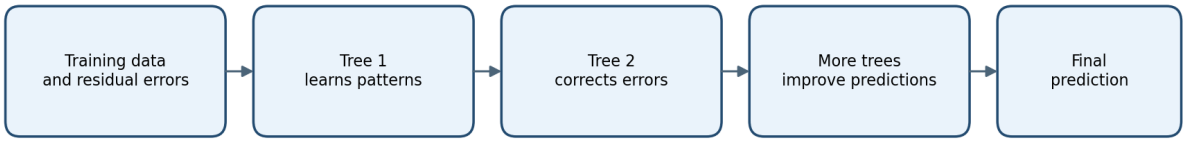
DAY 30
XGBoost: Modern Gradient Boosting

A detailed practical guide for the 60-Day Data Science Learning Journey

Simplified XGBoost boosting workflow



Each new tree contributes to the combined model. Regularization helps control complexity.

What you will learn

• Understand how XGBoost builds on gradient boosting.

• Train a classifier with XGBClassifier using a reproducible example.

• Evaluate predictions using accuracy, confusion matrix, precision, recall, and F1-score.

• Compare XGBoost with Random Forest and inspect feature importance.

• Avoid common errors and practice with worked solutions.

Prepared as a study resource. Model metrics depend on data and software environment.

1. Learning objectives

By the end of this lesson, you should be able to explain XGBoost in plain language, describe important hyperparameters, create a train/test split without leakage, train a binary classifier, interpret classification metrics, and explain why feature importance is useful but limited.

2. What is XGBoost?

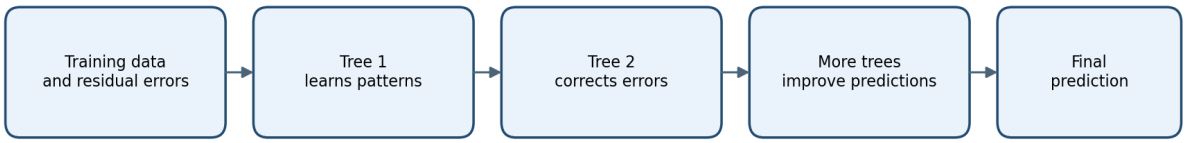
XGBoost means *Extreme Gradient Boosting*. It is an efficient gradient-boosted decision-tree implementation. It builds a sequence of trees, with each new tree helping improve the combined predictions made by earlier trees.

A decision tree splits data using rules such as “study hours greater than 3?” A boosting model combines many small trees. Rather than relying on one large tree, it adds trees step by step and optimizes a loss function. XGBoost includes regularization controls that discourage unnecessarily complex models.

A simple analogy

Imagine predicting whether a student will pass an exam. The first tree makes an initial prediction. The next tree focuses on patterns the current model has not handled well. Additional trees keep improving the combined model. The trees are not independent votes in the same way as Random Forest trees. Their contributions are added together.

Simplified XGBoost boosting workflow



Each new tree contributes to the combined model. Regularization helps control complexity.

Gradient boosting vs Random Forest

Aspect	Random Forest	Gradient Boosting / XGBoost
Main idea	Many trees trained with randomized samples/features	Trees added sequentially to improve the current model
Tree relationship	Trees can be trained largely independently	Later trees depend on the current ensemble's errors/gradients
Main risk	Can overfit, especially with deep trees	Can overfit if too complex or trained too long
Important controls	n_estimators, max_depth, max_features	learning_rate, n_estimators, max_depth, regularization

3. Why use XGBoost?

- **Strong tabular-data baseline:** often performs well on structured data such as sales, customer churn, and records.
- **Regularization:** supports penalties and split controls that limit complexity.
- **Flexible objectives:** supports classification and regression.
- **Missing values:** the tree algorithm can learn default directions for missing values in many workflows.
- **Diagnostics:** feature importance and evaluation tools help investigate the model.

> XGBoost is not automatically the best model. Performance depends on data quality, feature design, validation strategy, class balance, and parameter choices.

4. Important hyperparameters

Parameter	Meaning	Practical guidance
n_estimators	Maximum number of boosting rounds / trees	More trees can help, but may overfit or take longer.
learning_rate	How strongly each tree contributes	Smaller values often need more trees. Tune both together.
max_depth	Maximum tree depth	Deeper trees learn complex patterns but can overfit.
min_child_weight	Minimum sum of instance-weight/Hessian in a child	Higher values make splits more conservative.
subsample	Fraction of training rows sampled per round	Below 1.0 adds randomness and may reduce overfitting.
colsample_bytree	Fraction of features sampled for each tree	Below 1.0 can add diversity.
reg_lambda	L2 regularization on leaf weights	Higher values penalize large leaf weights.
reg_alpha	L1 regularization on leaf weights	Can encourage sparse leaf weights.
gamma	Minimum loss reduction needed for a split	Higher values require stronger evidence before splitting.
eval_metric	Metric used to evaluate model fit	For binary classification, logloss is a common choice.

Starter configuration: n_estimators=100, learning_rate=0.05, max_depth=3, min_child_weight=2, subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0. This is a starting point, not a universal best configuration.

5. Set up Python

```
python -m pip install xgboost pandas scikit-learn matplotlib
```

In a Jupyter cell, you can run: `!python -m pip install xgboost pandas scikit-learn matplotlib`. Ensure installation targets the same Python environment as your notebook.

Imports

```
import pandas as pd
import matplotlib.pyplot as plt

from xgboost import XGBClassifier, plot_importance
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)
```

6. Create a student pass/fail dataset

This example uses synthetic data with four input features. The target pass is 1 for pass and 0 for fail. Synthetic data teaches the workflow but does not establish real-world predictive validity.

```
data = {
    "study_hours": [
        1,2,3,4,5,6,7,2,3,5,6,1,4,7,8,
        2,5,6,3,8,4,1,7,5,2,6,8,3,4,7
    ],
    "attendance": [
        45,55,60,65,75,80,90,50,62,72,85,40,68,92,95,
        52,78,88,58,96,70,42,91,76,54,83,98,61,66,89
    ],
    "previous_score": [
        35,42,50,55,65,70,85,40,48,63,77,30,60,88,92,
        38,68,80,46,94,58,33,86,66,43,75,96,49,57,84
    ],
    "sleep_hours": [
        5,6,7,6,8,7,8,5,6,7,8,4,7,8,7,
        6,8,7,5,8,6,5,7,8,6,7,8,6,7,8
    ],
    "pass": [
        0,0,0,0,1,1,1,0,0,1,1,0,1,1,1,
        0,1,1,0,1,1,0,1,1,0,1,1,0,1,1
    ],
}
df = pd.DataFrame(data)
print(df.head())
print(df["pass"].value_counts())
```

Feature meanings

• study_hours: approximate hours spent studying.

• attendance: attendance percentage.

• previous_score: earlier exam score.

• sleep_hours: average sleep hours.

• pass: target label, 1 for pass and 0 for fail.

The feature columns are inputs X and the pass column is the target y.

7. Split the data

```
X = df[["study_hours", "attendance", "previous_score", "sleep_hours"]]
y = df["pass"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
```

test_size=0.25 assigns one quarter of rows to testing. random_state=42 makes the split reproducible. stratify=y attempts to preserve class proportions. Do not repeatedly tune on the test set. Use cross-validation on training data for model selection.

8. Train XGBoost

```
model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.05,
    max_depth=3,
    min_child_weight=2,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    eval_metric="logloss",
    random_state=42,
)
model.fit(X_train, y_train)
```

Line-by-line walkthrough

• XGBClassifier creates a classifier for a categorical target.

• n_estimators=100 allows up to 100 boosting rounds.

• learning_rate=0.05 makes each tree's contribution relatively small.

• max_depth=3 limits tree depth.

• min_child_weight=2 makes splitting more conservative than very small values.

• subsample=0.8 samples 80% of training rows per round.

• colsample_bytree=0.8 samples 80% of input features for each tree.

• reg_lambda=1.0 applies L2 regularization.

• eval_metric='logloss' uses log loss as the evaluation metric.

• random_state=42 helps make random operations reproducible.

• fit(X_train, y_train) trains on training features and labels.

9. Evaluate predictions

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

Metrics explained

Accuracy is the proportion of all predictions that are correct. It can mislead when classes are imbalanced.

Confusion matrix: with labels 0=fail and 1=pass, scikit-learn's usual binary layout is [[TN, FP], [FN, TP]]. TN = correctly predicted fail, FP = actual fail predicted pass, FN = actual pass predicted fail, TP = correctly predicted pass.

Precision for class 1 asks what fraction of predicted passes actually passed. **Recall for class 1** asks what fraction of actual passes were identified. **F1-score** combines precision and recall using their harmonic mean.

Choose metrics based on the real goal. If the goal is to identify students who need support, recall for the at-risk class may be more important than overall accuracy. Define the positive class carefully.

10. Predict a new student

```
new_student = pd.DataFrame([{
    "study_hours": 5,
    "attendance": 78,
    "previous_score": 67,
    "sleep_hours": 7,
}])

predicted_class = model.predict(new_student)[0]
predicted_probability = model.predict_proba(new_student)[0, 1]
print("Predicted class:", predicted_class)
print("Probability assigned to pass:", round(predicted_probability, 3))
```

The output depends on the trained model and data split. Probability is an estimate, not a guarantee. A tiny synthetic dataset is not suitable for real academic decisions.

11. Compare with Random Forest

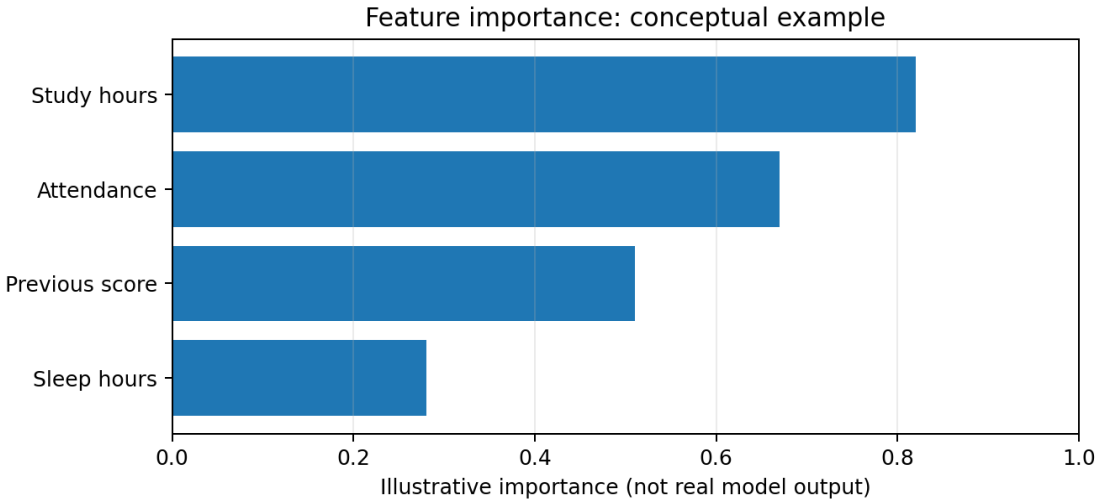
```
rf_model = RandomForestClassifier(
    n_estimators=200, max_depth=4, random_state=42
)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

print("XGBoost accuracy:", accuracy_score(y_test, y_pred))
print("Random Forest accuracy:", accuracy_score(y_test, rf_pred))
```

Use the same split for an initial comparison. Do not choose a winner from one tiny test set. Compare cross-validation, precision, recall, F1, calibration, speed, and maintenance needs.

12. Feature importance

```
ax = plot_importance(
    model, importance_type="gain", max_num_features=10, height=0.6
)
ax.set_title("XGBoost feature importance by gain")
plt.tight_layout()
plt.show()
```



The chart above is illustrative only, not output from the trained student model. The code creates the actual model-specific chart when run.

Gain summarizes objective improvement attributed to splits using a feature. Importance does not prove causation. Correlated features may share or compete for importance. For deeper interpretation, consider permutation importance, SHAP, and domain knowledge.

13. Hyperparameter tuning

• Start with a simple baseline and record your chosen metric.

• Use a validation split or cross-validation to select parameters.

• Tune learning_rate together with n_estimators.

• Try shallower trees or stronger regularization if training performance is high but validation performance is poor.

• Use early stopping with a clearly separated validation set and a compatible XGBoost API.

• Inspect class balance and consider weights or threshold adjustment for imbalanced tasks.

• Keep the final test set untouched until the end.

Example grid search

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [2, 3, 4],
    "learning_rate": [0.03, 0.05, 0.1],
    "min_child_weight": [1, 2, 4],
}
search = GridSearchCV(
    estimator=XGBClassifier(
        subsample=0.8, colsample_bytree=0.8,
        eval_metric="logloss", random_state=42
    ),
    param_grid=param_grid,
    scoring="f1",
    cv=cv,
    n_jobs=-1,
)
search.fit(X_train, y_train)
print("Best parameters:", search.best_params_)
print("Best CV F1:", search.best_score_)
best_model = search.best_estimator_
```

This grid has 3 × 3 × 3 × 3 = 81 combinations. Three-fold cross-validation means 243 fits. It can take time. Reduce the search or use RandomizedSearchCV for larger spaces. Scores from a tiny dataset remain uncertain.

14. Common errors

Symptom	Cause and fix
ModuleNotFoundError: xgboost	Install into the same Python environment: python -m pip install xgboost.
Unexpectedly high score	Check leakage, duplicates across splits, and synthetic target rules.
Label or class error	Inspect y.unique() and encode labels consistently, such as 0 and 1.
Feature order mismatch	Use the same named columns and feature schema at prediction time.
Metrics change between runs	Fix random_state where applicable. Small datasets make metrics variable.
Important feature interpreted as cause	Importance describes model usage, not causality. Check confounding and leakage.
Grid search is slow	Reduce combinations, use RandomizedSearchCV, or use fewer folds.

15. Exercises with complete solutions

Exercise 1: Change model settings

Task: Train with 200 trees, learning rate 0.03, and depth 2.

```
model_v2 = XGBClassifier(
    n_estimators=200, learning_rate=0.03, max_depth=2,
    min_child_weight=2, subsample=0.8, colsample_bytree=0.8,
```

```
    reg_lambda=1.0, eval_metric="logloss", random_state=42
)
model_v2.fit(X_train, y_train)
pred_v2 = model_v2.predict(X_test)
print(classification_report(y_test, pred_v2, zero_division=0))
```

Exercise 2: Calculate precision, recall, and F1

```
from sklearn.metrics import precision_score, recall_score, f1_score
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1:", f1_score(y_test, y_pred, zero_division=0))
```

Exercise 3: Rank features

```
importance = pd.Series(model.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False))
```

The highest value ranks first under the estimator's default importance definition. Rankings can change with the dataset, split, parameters, and importance type.

Exercise 4: Predict two students

```
new_students = pd.DataFrame([
    {"study_hours": 2, "attendance": 50, "previous_score": 40, "sleep_hours": 6},
    {"study_hours": 7, "attendance": 92, "previous_score": 88, "sleep_hours": 8},
])
classes = model.predict(new_students)
probabilities = model.predict_proba(new_students)[:, 1]
results = new_students.copy()
results["predicted_pass_class"] = classes
results["predicted_pass_probability"] = probabilities
print(results)
```

Exercise 5: Explain it to a non-technical person

Sample answer: “The model looks at examples with study time, attendance, previous scores, and sleep hours. It combines many small decision trees to estimate whether a student will pass. We test it on examples it did not train on. Its output is an estimate, not a guarantee, and it should not be used for important decisions until it has been validated on representative real data.”

16. Mini project extension

Replace the synthetic data with an ethically sourced, representative dataset. Document the source and permitted use. Define the target before exploring features. Split before fitting data-dependent preprocessing. Compare a majority-class baseline, logistic regression, Random Forest, and XGBoost. Use cross-validation for model selection. Report class-wise metrics and limitations. Never present a toy result as evidence about real students.

17. Revision notes

- XGBoost is an efficient gradient-boosted tree library.
- Boosting adds trees sequentially to improve the ensemble.
- Learning rate and number of trees should be considered together.
- Depth, min_child_weight, gamma, subsampling, and regularization control complexity.
- Use a held-out test set and metrics suited to the real task.
- Feature importance explains model usage, not causation.
- Tiny synthetic data teaches workflow, not real-world validity.

18. Quick knowledge check

- 1. What does XGBoost stand for?
- 2. How is boosting different from Random Forest?
- 3. What happens when learning_rate is reduced?
- 4. Why use stratify=y?

- 5. What do precision and recall measure?
- 6. Why does feature importance not prove causation?

Answer key

- 1. Extreme Gradient Boosting.
- 2. Boosting builds sequential trees. Random Forest combines trees trained with randomized samples/features.
- 3. Each tree contributes less, so more trees may be needed. Generalization may improve, but not guaranteed.
- 4. It attempts to preserve class proportions across training and test subsets.
- 5. Precision measures correctness among predicted positives. Recall measures the fraction of actual positives found.
- 6. The model can use correlations or confounded variables. Association is not causal proof.

End of Day 30. Save the notebook, note package versions, and record metrics only after running the code. Add this PDF alongside your Day 30 notebook or source code in your repository.